

## DÉPARTEMENT MATHÉMATIQUES ET INFORMATIQUE

Filière :

«Big Data et Cloud Computing»

### Projet AWS Academy

#### Mise en œuvre d'une Infrastructure Cloud de Supervision Centralisée sous AWS

Déploiement de Zabbix conteneurisé pour le monitoring  
d'un parc hybride (Linux & Windows)

Réalisé par :

EL BADRY Mohammed

Encadré par :

Pr. Azeddine KHIAT

**Technologies** : AWS EC2 • Docker • Zabbix 7.0 • MySQL • Ubuntu • Windows Server

Année Universitaire : 2025/2026

# Table des matières

|          |                                       |          |
|----------|---------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                   | <b>1</b> |
| 1.1      | Contexte du Projet                    | 1        |
| 1.2      | Objectifs du Projet                   | 1        |
| 1.3      | Technologies Utilisées                | 1        |
| 1.3.1    | Amazon Web Services (AWS)             | 1        |
| 1.3.2    | Docker                                | 2        |
| 1.3.3    | Zabbix                                | 2        |
| 1.4      | Architecture Cible                    | 2        |
| 1.5      | Dépôt GitHub                          | 2        |
| 1.6      | Organisation du Rapport               | 3        |
| <b>2</b> | <b>Architecture Réseau</b>            | <b>4</b> |
| 2.1      | Vue d'Ensemble                        | 4        |
| 2.2      | Schéma d'Architecture                 | 4        |
| 2.3      | Création du VPC                       | 5        |
| 2.3.1    | Configuration du VPC                  | 5        |
| 2.4      | Configuration du Subnet               | 5        |
| 2.4.1    | Subnet Public                         | 5        |
| 2.5      | Internet Gateway                      | 6        |
| 2.6      | Security Groups                       | 6        |
| 2.6.1    | Security Group - Serveur Zabbix       | 6        |
| 2.6.2    | Security Group - Clients              | 6        |
| 2.7      | Tableau Récapitulatif des Ports       | 7        |
| <b>3</b> | <b>Architecture des Instances EC2</b> | <b>8</b> |
| 3.1      | Vue d'Ensemble des Instances          | 8        |
| 3.2      | Serveur Zabbix                        | 8        |
| 3.2.1    | Spécifications                        | 8        |
| 3.2.2    | Justification du Choix t3.large       | 9        |
| 3.3      | Client Linux                          | 10       |
| 3.3.1    | Spécifications                        | 10       |
| 3.4      | Client Windows                        | 10       |
| 3.4.1    | Spécifications                        | 10       |
| 3.4.2    | Récupération du Mot de Passe Windows  | 11       |
| 3.5      | Connexion aux Instances               | 12       |
| 3.5.1    | Connexion SSH au Serveur Zabbix       | 12       |
| 3.5.2    | Connexion RDP au Client Windows       | 12       |
| 3.6      | Tableau Récapitulatif                 | 13       |

|          |                                           |           |
|----------|-------------------------------------------|-----------|
| <b>4</b> | <b>Déploiement du Serveur Zabbix</b>      | <b>14</b> |
| 4.1      | Installation de Docker                    | 14        |
| 4.1.1    | Mise à jour du Système                    | 14        |
| 4.1.2    | Installation de Docker Engine             | 14        |
| 4.1.3    | Vérification de l'Installation            | 15        |
| 4.2      | Configuration de Zabbix                   | 15        |
| 4.2.1    | Création du Fichier docker-compose.yml    | 15        |
| 4.3      | Lancement des Conteneurs                  | 17        |
| 4.3.1    | Démarrage de la Stack                     | 17        |
| 4.3.2    | Vérification des Conteneurs               | 17        |
| 4.4      | Accès à l'Interface Web                   | 18        |
| 4.4.1    | Connexion à Zabbix                        | 18        |
| 4.4.2    | Dashboard Principal                       | 18        |
| 4.5      | Architecture des Conteneurs               | 19        |
| 4.6      | Commandes Utiles                          | 19        |
| <b>5</b> | <b>Configuration des Clients (Agents)</b> | <b>20</b> |
| 5.1      | Vue d'Ensemble                            | 20        |
| 5.2      | Installation sur Linux (Ubuntu)           | 20        |
| 5.2.1    | Connexion au Client Linux                 | 20        |
| 5.2.2    | Installation de l'Agent Zabbix            | 20        |
| 5.2.3    | Configuration de l'Agent Linux            | 20        |
| 5.2.4    | Démarrage du Service                      | 21        |
| 5.2.5    | Vérification de la Connectivité           | 21        |
| 5.3      | Installation sur Windows                  | 22        |
| 5.3.1    | Connexion RDP                             | 22        |
| 5.3.2    | Téléchargement de l'Agent                 | 22        |
| 5.3.3    | Installation de l'Agent Windows           | 23        |
| 5.3.4    | Vérification du Service Windows           | 23        |
| 5.3.5    | Configuration du Pare-feu Windows         | 23        |
| 5.4      | Ajout des Hôtes dans Zabbix               | 24        |
| 5.4.1    | Ajouter le Client Linux                   | 24        |
| 5.4.2    | Ajouter le Client Windows                 | 24        |
| 5.5      | Vérification de la Connexion              | 25        |
| 5.5.1    | Liste des Hôtes                           | 25        |
| 5.5.2    | Statut des Agents (ZBX Vert)              | 25        |
| 5.6      | Tableau Récapitulatif des Agents          | 25        |
| <b>6</b> | <b>Monitoring et Tableaux de Bord</b>     | <b>26</b> |
| 6.1      | Vue d'Ensemble du Monitoring              | 26        |
| 6.2      | Dashboard Principal                       | 26        |
| 6.3      | Métriques Collectées                      | 26        |
| 6.3.1    | Métriques Système Linux                   | 26        |
| 6.3.2    | Métriques Système Windows                 | 27        |
| 6.4      | Test de Charge (Stress Test)              | 27        |
| 6.4.1    | Objectif                                  | 27        |
| 6.4.2    | Exécution du Test                         | 27        |
| 6.4.3    | Observation dans Zabbix                   | 28        |
| 6.5      | Gestion des Alertes                       | 28        |

|          |                                       |           |
|----------|---------------------------------------|-----------|
| 6.5.1    | Problèmes Détectés . . . . .          | 28        |
| 6.5.2    | Liste des Problèmes . . . . .         | 29        |
| 6.5.3    | Historique des Événements . . . . .   | 29        |
| 6.6      | Configuration des Triggers . . . . .  | 30        |
| 6.6.1    | Liste des Triggers . . . . .          | 30        |
| 6.6.2    | Exemples de Triggers . . . . .        | 31        |
| 6.7      | Visualisation des Données . . . . .   | 32        |
| 6.7.1    | Latest Data . . . . .                 | 32        |
| 6.7.2    | Graphiques . . . . .                  | 32        |
| 6.8      | Récapitulatif du Monitoring . . . . . | 32        |
| 6.9      | Points Clés . . . . .                 | 32        |
| <b>7</b> | <b>Conclusion</b>                     | <b>33</b> |
| 7.1      | Synthèse du Projet . . . . .          | 33        |
| 7.2      | Compétences Acquises . . . . .        | 33        |
| 7.2.1    | Amazon Web Services . . . . .         | 33        |
| 7.2.2    | Docker et Conteneurisation . . . . .  | 33        |
| 7.2.3    | Zabbix et Monitoring . . . . .        | 34        |
| 7.3      | Difficultés Rencontrées . . . . .     | 34        |
| 7.3.1    | Problèmes et Solutions . . . . .      | 34        |
| 7.4      | Limitations du Learner Lab . . . . .  | 34        |
| 7.5      | Améliorations Possibles . . . . .     | 34        |
| 7.6      | Perspectives . . . . .                | 35        |
| 7.7      | Conclusion Générale . . . . .         | 35        |

# Table des figures

|     |                                                          |    |
|-----|----------------------------------------------------------|----|
| 2.1 | Architecture réseau du projet AWS Zabbix . . . . .       | 4  |
| 2.2 | Création du VPC dans la console AWS . . . . .            | 5  |
| 2.3 | Configuration du Subnet public . . . . .                 | 6  |
| 3.1 | Liste des instances EC2 en état Running . . . . .        | 8  |
| 3.2 | Détails de l'instance Zabbix Server . . . . .            | 9  |
| 3.3 | Détails de l'instance Client Linux . . . . .             | 10 |
| 3.4 | Détails de l'instance Client Windows . . . . .           | 11 |
| 3.5 | Récupération du mot de passe Windows . . . . .           | 11 |
| 3.6 | Connexion SSH réussie au serveur Zabbix . . . . .        | 12 |
| 3.7 | Connexion RDP au client Windows . . . . .                | 13 |
| 4.1 | Installation de Docker réussie . . . . .                 | 15 |
| 4.2 | Lancement des conteneurs Docker . . . . .                | 17 |
| 4.3 | Conteneurs Zabbix en cours d'exécution . . . . .         | 18 |
| 4.4 | Page de connexion Zabbix . . . . .                       | 18 |
| 4.5 | Dashboard principal de Zabbix . . . . .                  | 19 |
| 4.6 | Architecture des conteneurs Docker . . . . .             | 19 |
| 5.1 | Configuration de l'agent Zabbix sur Linux . . . . .      | 21 |
| 5.2 | Connexion RDP au client Windows . . . . .                | 22 |
| 5.3 | Installation de l'agent Zabbix sur Windows . . . . .     | 23 |
| 5.4 | Ajout du client Linux dans Zabbix . . . . .              | 24 |
| 5.5 | Liste des hôtes configurés dans Zabbix . . . . .         | 25 |
| 5.6 | Statut ZBX vert - Agents connectés avec succès . . . . . | 25 |
| 6.1 | Dashboard de monitoring Zabbix . . . . .                 | 26 |
| 6.2 | Exécution de la commande stress . . . . .                | 28 |
| 6.3 | Pic de CPU détecté suite au stress test . . . . .        | 28 |
| 6.4 | Vue des problèmes actifs dans Zabbix . . . . .           | 29 |
| 6.5 | Liste détaillée des problèmes . . . . .                  | 29 |
| 6.6 | Historique des événements Zabbix . . . . .               | 30 |
| 6.7 | Liste des triggers configurés . . . . .                  | 30 |
| 6.8 | Configuration d'un trigger . . . . .                     | 31 |

# Liste des tableaux

|     |                                                  |    |
|-----|--------------------------------------------------|----|
| 1.1 | Services AWS utilisés . . . . .                  | 2  |
| 2.1 | Configuration du VPC . . . . .                   | 5  |
| 2.2 | Configuration du Subnet . . . . .                | 5  |
| 2.3 | Règles du Security Group Zabbix Server . . . . . | 6  |
| 2.4 | Règles du Security Group Clients . . . . .       | 6  |
| 2.5 | Ports utilisés dans l'infrastructure . . . . .   | 7  |
| 3.1 | Récapitulatif des instances EC2 . . . . .        | 8  |
| 3.2 | Configuration du serveur Zabbix . . . . .        | 9  |
| 3.3 | Configuration du client Linux . . . . .          | 10 |
| 3.4 | Configuration du client Windows . . . . .        | 10 |
| 3.5 | Récapitulatif complet des instances . . . . .    | 13 |
| 4.1 | Commandes Docker utiles . . . . .                | 19 |
| 5.1 | Modes de communication des agents . . . . .      | 20 |
| 5.2 | Configuration des agents . . . . .               | 25 |
| 6.1 | Métriques Linux collectées . . . . .             | 27 |
| 6.2 | Métriques Windows collectées . . . . .           | 27 |
| 6.3 | Exemples de triggers Zabbix . . . . .            | 31 |
| 6.4 | État final du monitoring . . . . .               | 32 |
| 7.1 | Objectifs atteints . . . . .                     | 33 |
| 7.2 | Difficultés et solutions apportées . . . . .     | 34 |

# Chapitre 1

## Introduction

### 1.1 Contexte du Projet

Dans le cadre de la formation AWS Academy, ce projet vise à mettre en œuvre une **infrastructure cloud de supervision centralisée** sur Amazon Web Services (AWS). L'objectif principal est de déployer une solution de monitoring complète capable de surveiller un parc informatique hybride composé de machines Linux et Windows.

La supervision des infrastructures informatiques est devenue un enjeu majeur pour les entreprises. Elle permet de :

- Détecter les anomalies et les pannes en temps réel
- Anticiper les problèmes de performance
- Optimiser l'utilisation des ressources
- Garantir la disponibilité des services

### 1.2 Objectifs du Projet

Ce projet a pour objectifs :

1. **Déployer une infrastructure AWS** : Création d'un VPC, configuration des sous-réseaux et des groupes de sécurité
2. **Installer Zabbix avec Docker** : Mise en place d'un serveur Zabbix conteneurisé pour la centralisation du monitoring
3. **Configurer la supervision hybride** : Installation et configuration des agents Zabbix sur des clients Linux et Windows
4. **Valider le monitoring** : Démonstration de la collecte de métriques et de la génération d'alertes

### 1.3 Technologies Utilisées

#### 1.3.1 Amazon Web Services (AWS)

AWS est la plateforme cloud leader du marché, offrant plus de 200 services. Pour ce projet, nous utilisons :

TABLE 1.1 – Services AWS utilisés

| Service         | Description                                                  |
|-----------------|--------------------------------------------------------------|
| EC2             | Instances virtuelles pour héberger le serveur et les clients |
| VPC             | Réseau virtuel privé isolé                                   |
| Security Groups | Pare-feu virtuel pour contrôler le trafic                    |

### 1.3.2 Docker

Docker est une plateforme de conteneurisation qui permet de :

- Empaqueter les applications avec leurs dépendances
- Garantir la portabilité et la reproductibilité
- Simplifier le déploiement et la mise à jour

### 1.3.3 Zabbix

Zabbix est une solution de monitoring open source de niveau entreprise, capable de surveiller :

- Les métriques système (CPU, RAM, disque, réseau)
- Les services et applications
- Les équipements réseau
- Les environnements cloud

## 1.4 Architecture Cible

L'architecture déployée comprend :

- **1 VPC** avec un sous-réseau public (10.0.0.0/16)
- **3 instances EC2** :
  - Serveur Zabbix (t3.large) - Ubuntu avec Docker
  - Client Linux (t3.medium) - Ubuntu avec agent Zabbix
  - Client Windows (t3.large) - Windows Server avec agent Zabbix
- **Security Groups** configurés pour les ports nécessaires

## 1.5 Dépôt GitHub

L'ensemble du code source, des fichiers de configuration et de la documentation est disponible sur GitHub :

#### Dépôt GitHub du projet

<https://github.com/VOTRE-USERNAME/aws-zabbix-monitoring>



## 1.6 Organisation du Rapport

Ce rapport est structuré comme suit :

1. **Chapitre 2** : Architecture réseau et configuration du VPC
2. **Chapitre 3** : Création et configuration des instances EC2
3. **Chapitre 4** : Déploiement de Zabbix avec Docker
4. **Chapitre 5** : Configuration des agents sur les clients
5. **Chapitre 6** : Monitoring et tableaux de bord
6. **Chapitre 7** : Conclusion et perspectives

# Chapitre 2

## Architecture Réseau

### 2.1 Vue d'Ensemble

L'architecture réseau déployée sur AWS repose sur un Virtual Private Cloud (VPC) configuré pour héberger l'ensemble de l'infrastructure de monitoring. Cette architecture a été conçue en tenant compte des limitations du Learner Lab AWS Academy.

### 2.2 Schéma d'Architecture

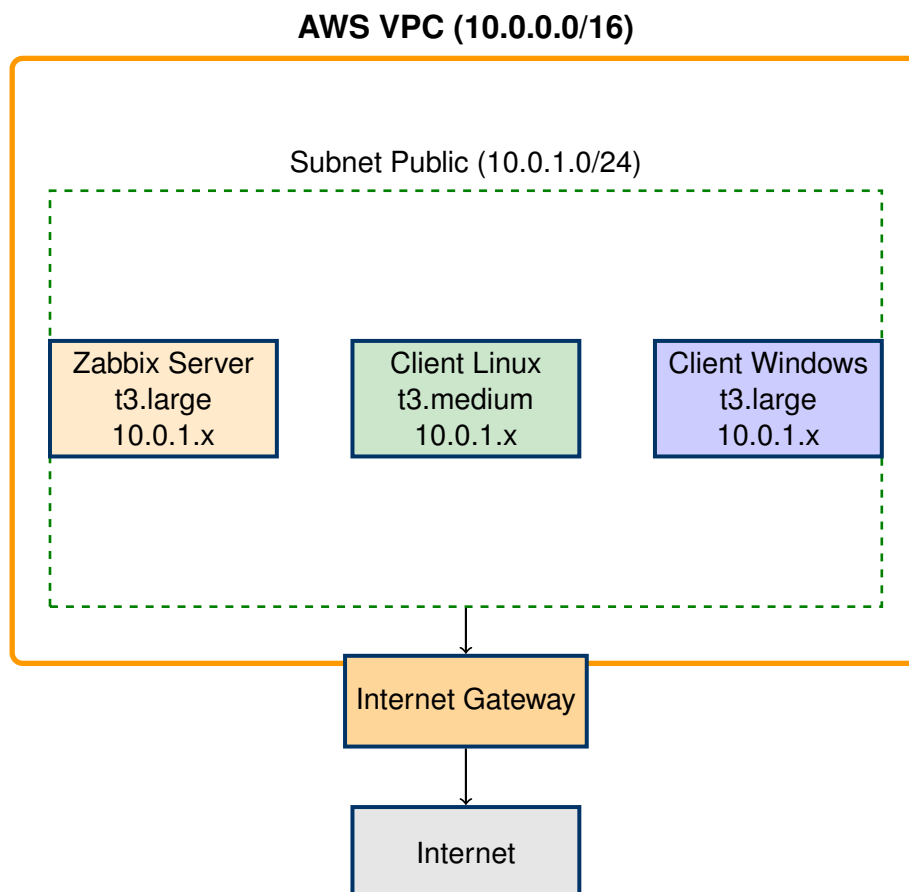


FIGURE 2.1 – Architecture réseau du projet AWS Zabbix

## 2.3 Création du VPC

### 2.3.1 Configuration du VPC

Le VPC a été créé avec les paramètres suivants :

TABLE 2.1 – Configuration du VPC

| Paramètre | Valeur             |
|-----------|--------------------|
| Nom       | ELBadry-Zabbix-VPC |
| IPv4 CIDR | 10.0.0.0/16        |
| IPv6      | Désactivé          |
| Tenancy   | Default            |

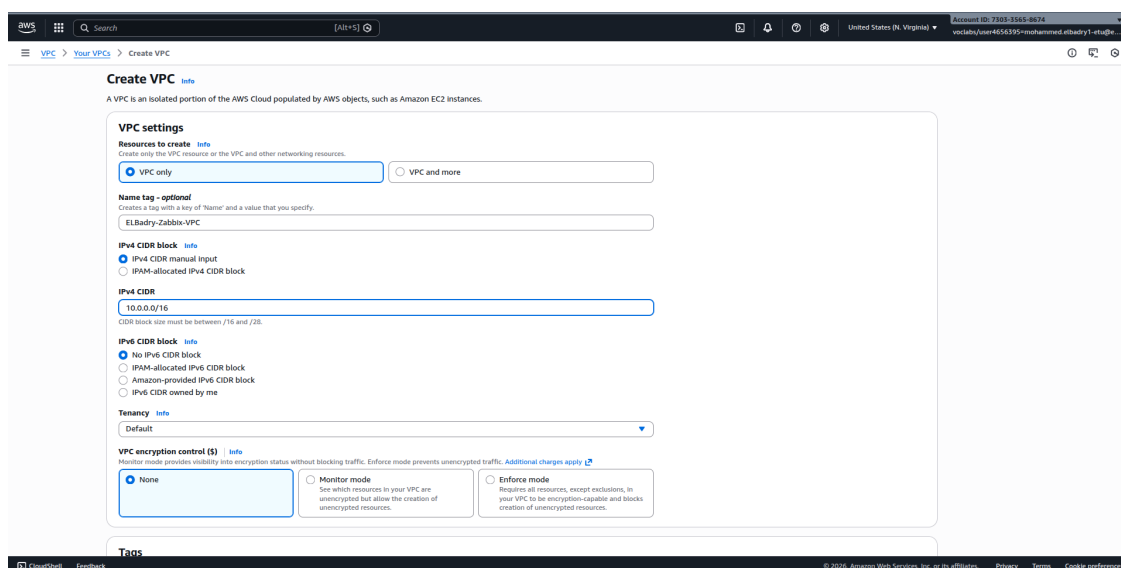


FIGURE 2.2 – Création du VPC dans la console AWS

## 2.4 Configuration du Subnet

### 2.4.1 Subnet Public

Un sous-réseau public a été créé pour héberger toutes les instances :

TABLE 2.2 – Configuration du Subnet

| Paramètre             | Valeur                |
|-----------------------|-----------------------|
| Nom                   | ELBadry-Public-Subnet |
| VPC                   | ELBadry-Zabbix-VPC    |
| Availability Zone     | us-east-1a            |
| IPv4 CIDR             | 10.0.1.0/24           |
| Auto-assign Public IP | Activé                |

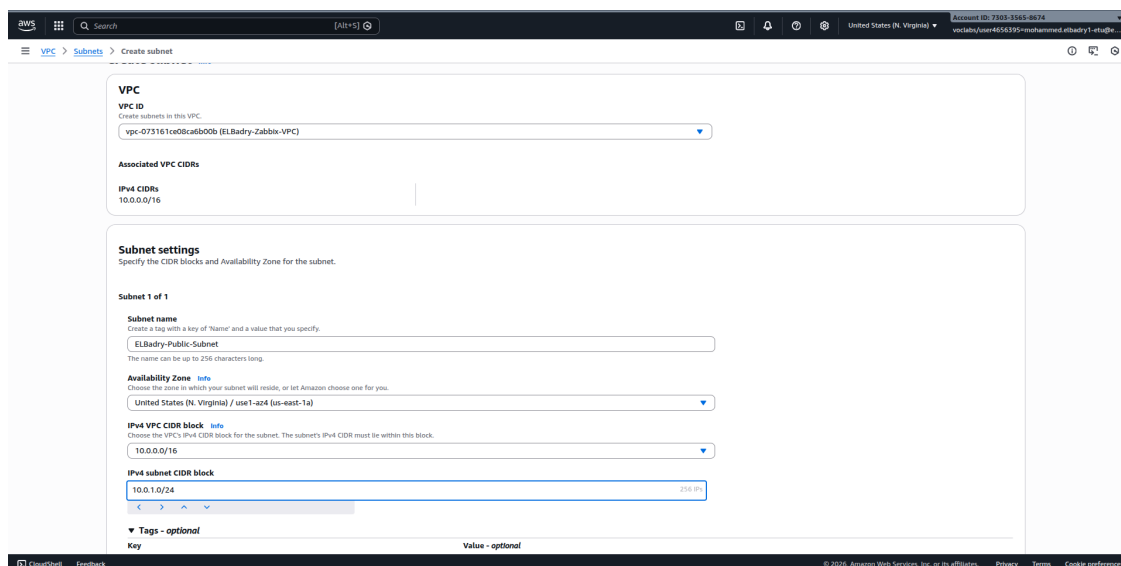


FIGURE 2.3 – Configuration du Subnet public

## 2.5 Internet Gateway

L'Internet Gateway permet aux instances du VPC de communiquer avec Internet :

1. Création de l'Internet Gateway : ELBdry-IGW
2. Attachement au VPC ELBdry-Zabbix-VPC
3. Configuration de la table de routage avec la route 0.0.0.0/0 vers l'IGW

## 2.6 Security Groups

### 2.6.1 Security Group - Serveur Zabbix

TABLE 2.3 – Règles du Security Group Zabbix Server

| Type       | Port  | Source      | Description             |
|------------|-------|-------------|-------------------------|
| SSH        | 22    | Mon IP      | Accès SSH               |
| HTTP       | 80    | 0.0.0.0/0   | Interface Web Zabbix    |
| HTTPS      | 443   | 0.0.0.0/0   | Interface Web sécurisée |
| Custom TCP | 10050 | 10.0.0.0/16 | Zabbix Agent            |
| Custom TCP | 10051 | 10.0.0.0/16 | Zabbix Trapper          |

### 2.6.2 Security Group - Clients

TABLE 2.4 – Règles du Security Group Clients

| Type       | Port  | Source           | Description                 |
|------------|-------|------------------|-----------------------------|
| SSH        | 22    | Mon IP           | Accès SSH (Linux)           |
| RDP        | 3389  | Mon IP           | Bureau à distance (Windows) |
| Custom TCP | 10050 | SG Zabbix Server | Agent Zabbix                |

## 2.7 Tableau Récapitulatif des Ports

TABLE 2.5 – Ports utilisés dans l’infrastructure

| Port  | Service        | Description                             |
|-------|----------------|-----------------------------------------|
| 22    | SSH            | Accès sécurisé aux machines Linux       |
| 80    | HTTP           | Interface Web Zabbix                    |
| 443   | HTTPS          | Interface Web Zabbix (sécurisé)         |
| 3389  | RDP            | Bureau à distance Windows               |
| 10050 | Zabbix Agent   | Communication passive (serveur → agent) |
| 10051 | Zabbix Trapper | Communication active (agent → serveur)  |

# Chapitre 3

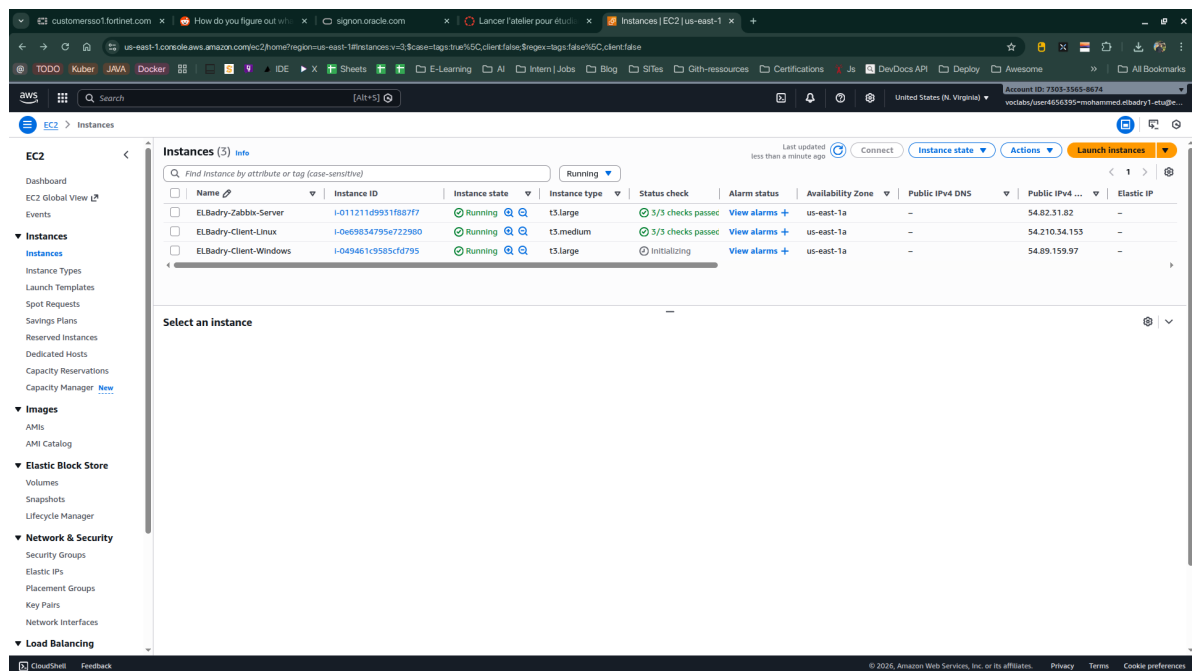
## Architecture des Instances EC2

### 3.1 Vue d'Ensemble des Instances

Trois instances EC2 ont été déployées pour ce projet, chacune ayant un rôle spécifique dans l'infrastructure de monitoring.

TABLE 3.1 – Récapitulatif des instances EC2

| Instance               | Type      | OS                  | Rôle                    |
|------------------------|-----------|---------------------|-------------------------|
| ELBadry-Zabbix-Server  | t3.large  | Ubuntu 22.04        | Serveur Zabbix (Docker) |
| ELBadry-Client-Linux   | t3.medium | Ubuntu 22.04        | Client supervisé        |
| ELBadry-Client-Windows | t3.large  | Windows Server 2022 | Client supervisé        |



| Name                   | Instance ID         | Instance state | Instance type | Status check      | Alarm status | Availability Zone | Public IPv4 DNS | Public IPv4 ... | Elastic IP |
|------------------------|---------------------|----------------|---------------|-------------------|--------------|-------------------|-----------------|-----------------|------------|
| ELBadry-Zabbix-Server  | i-011211d9951f887f7 | Running        | t3.large      | 3/5 checks passed | View alarms  | us-east-1a        | -               | 54.82.31.82     | -          |
| ELBadry-Client-Linux   | i-0e69854795e722560 | Running        | t3.medium     | 3/5 checks passed | View alarms  | us-east-1a        | -               | 54.210.34.153   | -          |
| ELBadry-Client-Windows | i-049461c9585cf795  | Initializing   | t3.large      | Initializing      | View alarms  | us-east-1a        | -               | 54.89.159.97    | -          |

FIGURE 3.1 – Liste des instances EC2 en état Running

### 3.2 Serveur Zabbix

#### 3.2.1 Spécifications

Le serveur Zabbix est le cœur de l'infrastructure de monitoring :

TABLE 3.2 – Configuration du serveur Zabbix

| Paramètre       | Valeur                      |
|-----------------|-----------------------------|
| Nom             | ELBadry-Zabbix-Server       |
| Type d'instance | t3.large (2 vCPU, 8 Go RAM) |
| AMI             | Ubuntu Server 22.04 LTS     |
| Stockage        | 30 Go gp3                   |
| IP Privée       | 10.0.1.x                    |
| Security Group  | ELBadry-Zabbix-Server-SG    |

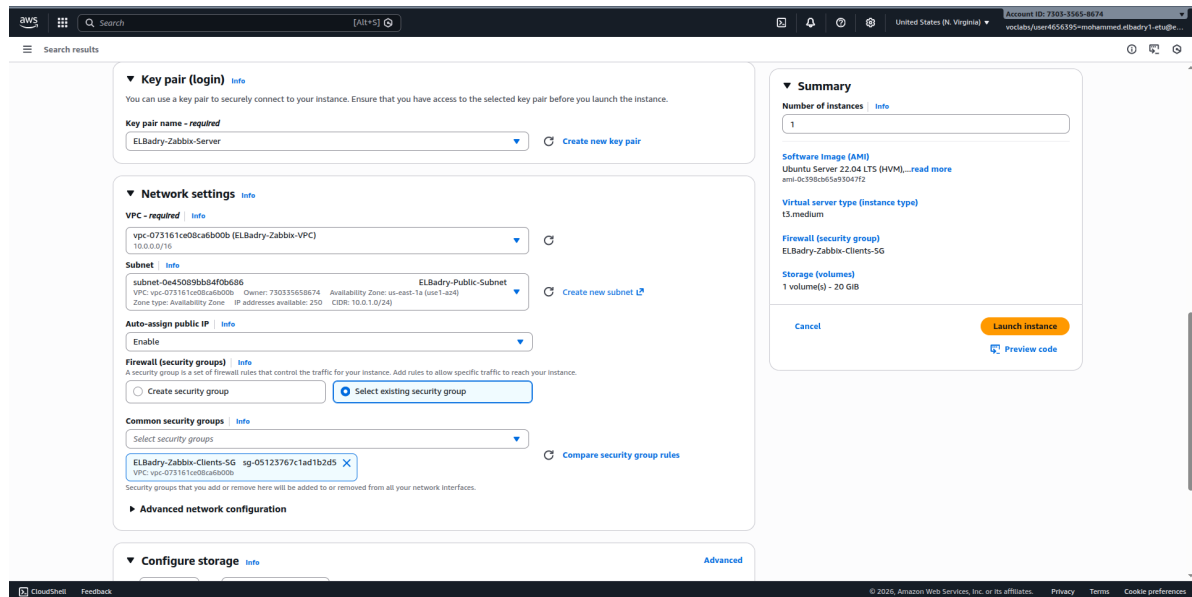


FIGURE 3.2 – Détails de l'instance Zabbix Server

### 3.2.2 Justification du Choix t3.large

Le type t3.large a été choisi pour le serveur Zabbix car :

- **8 Go de RAM** : Suffisant pour Docker, MySQL et Zabbix Server
- **2 vCPU** : Permet de gérer plusieurs conteneurs simultanément
- **Performances en rafale** : Adapté aux pics de charge
- **Compatible Learner Lab** : Type d'instance autorisé

## 3.3 Client Linux

### 3.3.1 Spécifications

TABLE 3.3 – Configuration du client Linux

| Paramètre       | Valeur                       |
|-----------------|------------------------------|
| Nom             | ELBadry-Client-Linux         |
| Type d'instance | t3.medium (2 vCPU, 4 Go RAM) |
| AMI             | Ubuntu Server 22.04 LTS      |
| Stockage        | 20 Go gp3                    |
| IP Privée       | 10.0.1.x                     |
| Security Group  | ELBadry-Zabbix-Clients-SG    |

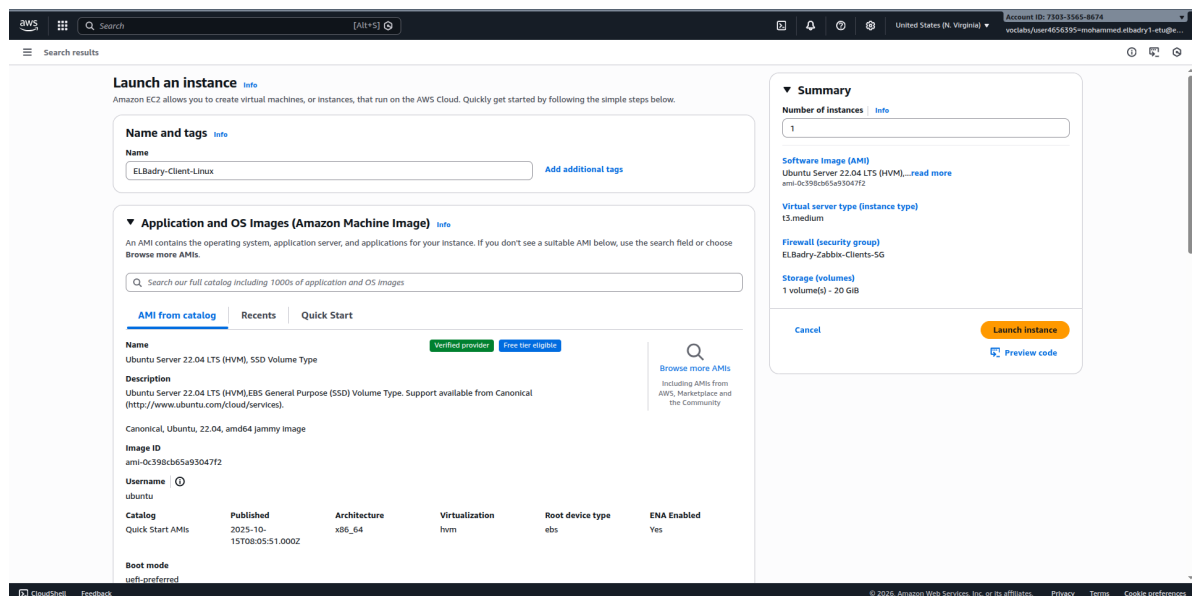


FIGURE 3.3 – Détails de l'instance Client Linux

## 3.4 Client Windows

### 3.4.1 Spécifications

TABLE 3.4 – Configuration du client Windows

| Paramètre       | Valeur                             |
|-----------------|------------------------------------|
| Nom             | ELBadry-Client-Windows             |
| Type d'instance | t3.large (2 vCPU, 8 Go RAM)        |
| AMI             | Microsoft Windows Server 2022 Base |
| Stockage        | 30 Go gp3                          |
| IP Privée       | 10.0.1.x                           |
| Security Group  | ELBadry-Zabbix-Clients-SG          |



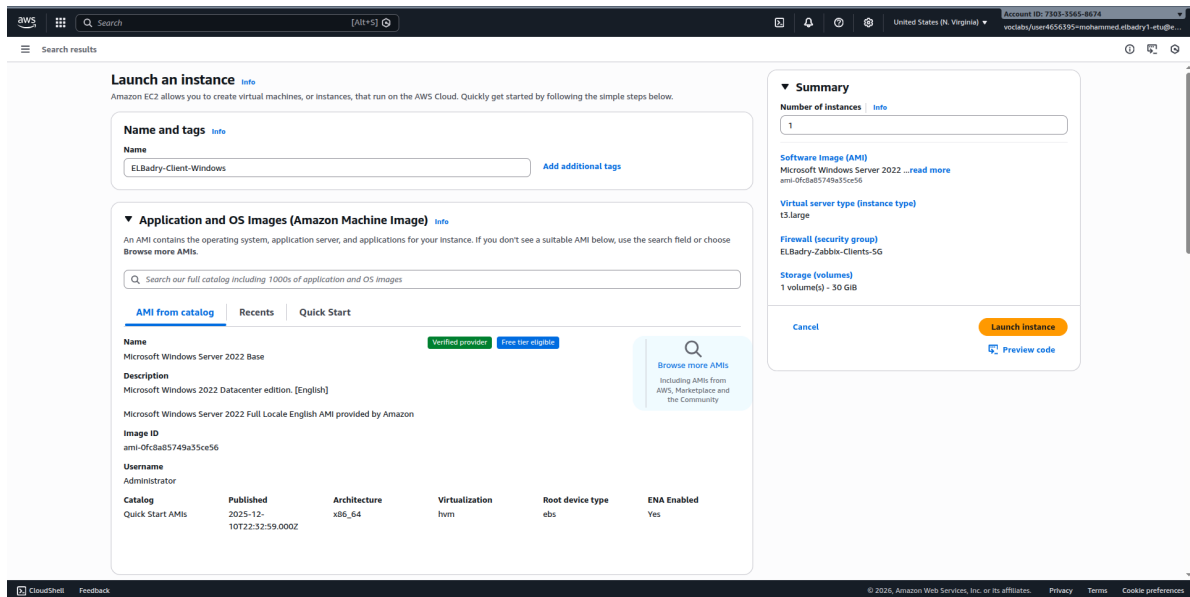


FIGURE 3.4 – Détails de l'instance Client Windows

### 3.4.2 Récupération du Mot de Passe Windows

Pour accéder au client Windows via RDP, il est nécessaire de récupérer le mot de passe administrateur :

1. Sélectionner l'instance Windows
2. Actions → Security → Get Windows Password
3. Charger la clé privée (.pem)
4. Déchiffrer le mot de passe

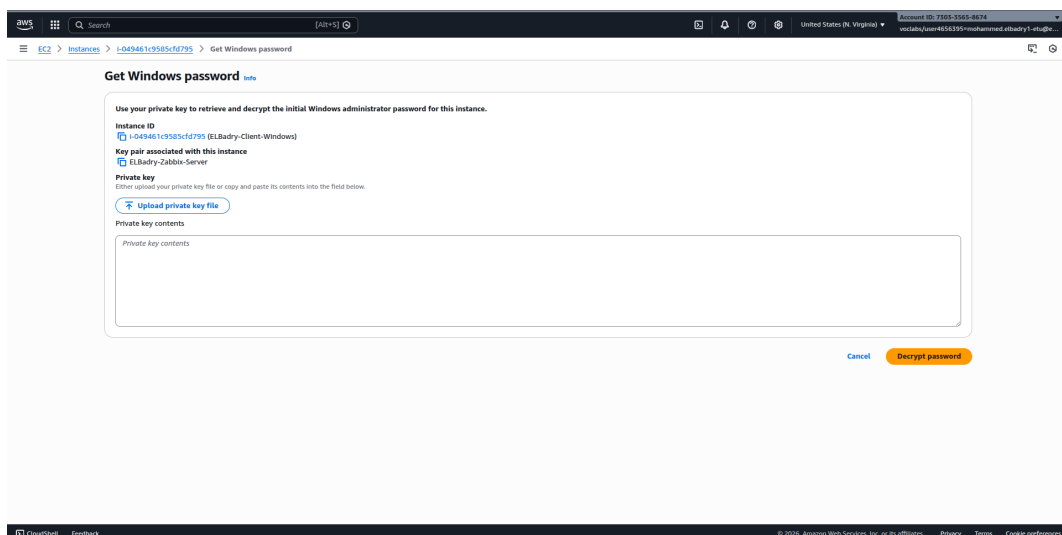


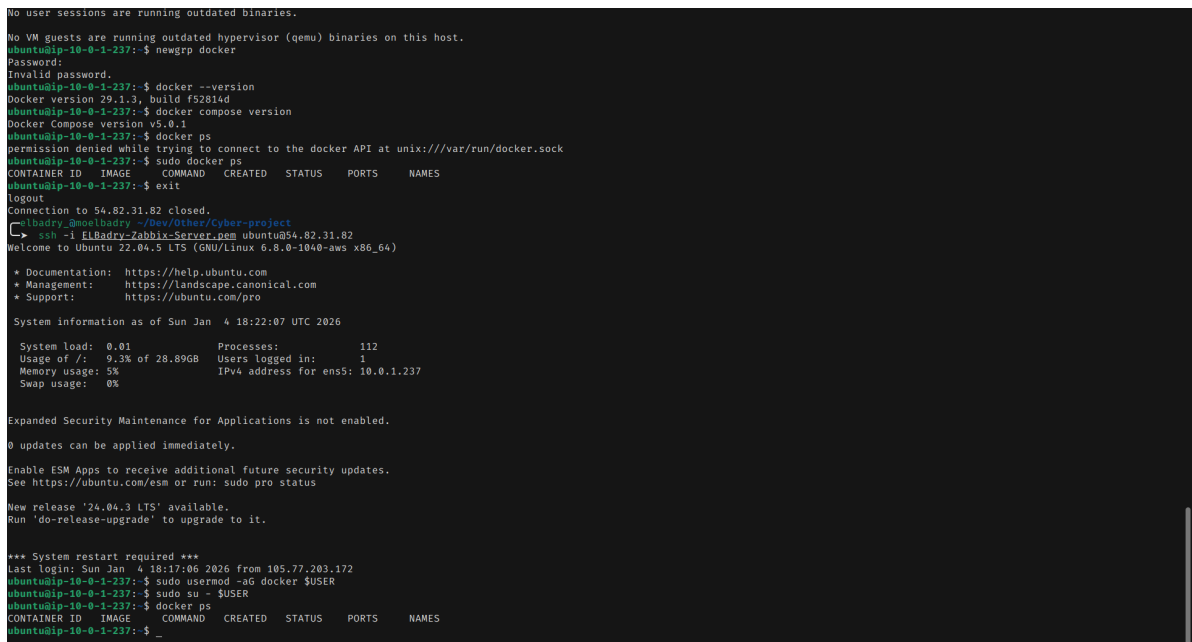
FIGURE 3.5 – Récupération du mot de passe Windows

## 3.5 Connexion aux Instances

### 3.5.1 Connexion SSH au Serveur Zabbix

```
1 ssh -i "votre-cle.pem" ubuntu@<IP-PUBLIQUE-ZABBIX>
```

Code 3.1 – Connexion SSH au serveur Zabbix



```
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-237:~$ newgrp docker
Password:
Invalid password.
ubuntu@ip-10-0-1-237:~$ docker --version
Docker version 29.1.3, build f52814d
ubuntu@ip-10-0-1-237:~$ docker compose version
Docker Compose version v5.0.1
ubuntu@ip-10-0-1-237:~$ docker ps
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
ubuntu@ip-10-0-1-237:~$ exit
logout
Connection to 54.82.31.82 closed.
❯ hadry_@macbookpro:~/dev/Other/Cyber-project
❯ ssh -i ELBadry-Zabbix-Server.pem ubuntu@54.82.31.82
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1040-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sun Jan  4 18:22:07 UTC 2026

System load:  0.01               Processes:    112
Usage of /:   9.3% of 28.89GB     Users logged in: 1
Memory usage: 5M                 IPv4 address for ens5: 10.0.1.237
Swap usage:   0K

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Sun Jan  4 18:17:06 2026 from 105.77.203.172
ubuntu@ip-10-0-1-237:~$ sudo usermod -sG docker $USER
ubuntu@ip-10-0-1-237:~$ sudo su - $USER
ubuntu@ip-10-0-1-237:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED   STATUS    PORTS   NAMES
ubuntu@ip-10-0-1-237:~$
```

FIGURE 3.6 – Connexion SSH réussie au serveur Zabbix

### 3.5.2 Connexion RDP au Client Windows

1. Ouvrir le client Bureau à distance (mstsc)
2. Entrer l'IP publique de l'instance Windows
3. Utiliser le nom d'utilisateur : Administrator
4. Entrer le mot de passe déchiffré

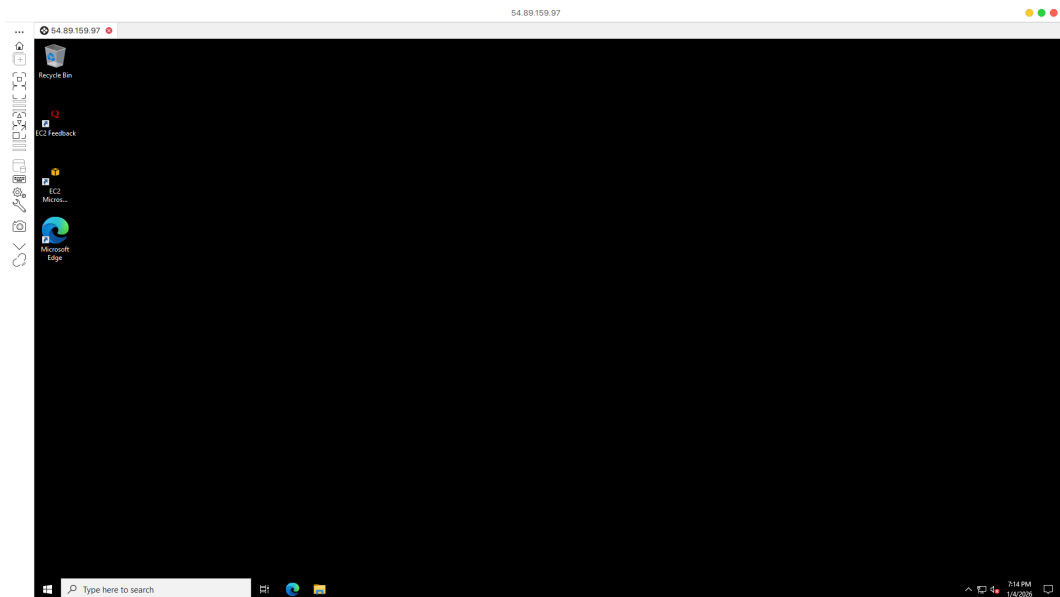


FIGURE 3.7 – Connexion RDP au client Windows

### 3.6 Tableau Récapitulatif

TABLE 3.5 – Récapitulatif complet des instances

| Propriété | Zabbix Server | Client Linux | Client Windows      |
|-----------|---------------|--------------|---------------------|
| Type      | t3.large      | t3.medium    | t3.large            |
| vCPU      | 2             | 2            | 2                   |
| RAM       | 8 Go          | 4 Go         | 8 Go                |
| OS        | Ubuntu 22.04  | Ubuntu 22.04 | Windows Server 2022 |
| Stockage  | 30 Go         | 20 Go        | 30 Go               |
| Accès     | SSH (22)      | SSH (22)     | RDP (3389)          |

# Chapitre 4

## Déploiement du Serveur Zabbix

### 4.1 Installation de Docker

#### 4.1.1 Mise à jour du Système

Après connexion au serveur Zabbix via SSH, la première étape consiste à mettre à jour le système :

```
1 sudo apt update && sudo apt upgrade -y
```

Code 4.1 – Mise à jour du système

#### 4.1.2 Installation de Docker Engine

Installation de Docker et Docker Compose :

```
1 # Installation des prerequis
2 sudo apt install -y ca-certificates curl gnupg
3
4 # Ajout de la cle GPG Docker
5 sudo install -m 0755 -d /etc/apt/keyrings
6 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
7     sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
8
9 # Ajout du repository Docker
10 echo "deb [arch=$(dpkg --print-architecture) \
11     signed-by=/etc/apt/keyrings/docker.gpg] \
12     https://download.docker.com/linux/ubuntu \
13     $(lsb_release -cs) stable" | \
14     sudo tee /etc/apt/sources.list.d/docker.list
15
16 # Installation de Docker
17 sudo apt update
18 sudo apt install -y docker-ce docker-ce-cli \
19     containerd.io docker-compose-plugin
20
21 # Ajout de l'utilisateur au groupe docker
22 sudo usermod -aG docker $USER
```

Code 4.2 – Installation de Docker

```
ubuntu@ip-10-0-1-237:~$ sudo usermod -aG docker $USER
ubuntu@ip-10-0-1-237:~$ sudo su - $USER
ubuntu@ip-10-0-1-237:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-10-0-1-237:~$ mkdir -p ~/zabbix
cd ~/zabbix
ubuntu@ip-10-0-1-237:~/zabbix$ nano docker-compose.yml
ubuntu@ip-10-0-1-237:~/zabbix$ l
docker-compose.yml
ubuntu@ip-10-0-1-237:~/zabbix$ cat docker-compose.yml
version: "3.8"

services:
  # Base de données MySQL
  mysql-server:
    image: mysql:8.0
    container_name: zabbix-mysql
    restart: always
    environment:
      MYSQL_DATABASE: zabbix
      MYSQL_USER: zabbix
      MYSQL_PASSWORD: zabbix_pwd_secure
      MYSQL_ROOT_PASSWORD: root_pwd_secure
    command:
      - mysqld
      - --character-set-server=utf8mb4
      - --collation-server=utf8mb4_bin
      - --default-authentication-plugin=mysql_native_password
    volumes:
      - mysql_data:/var/lib/mysql
    networks:
      - zabbix-net
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Serveur Zabbix
  zabbix-server:
    image: zabbix/zabbix-server-mysql:alpine-7.0-latest
    container_name: zabbix-server
    restart: always
    environment:
      DB_SERVER_HOST: mysql-server
```

FIGURE 4.1 – Installation de Docker réussie

### 4.1.3 Vérification de l'Installation

```
1 docker --version
2 docker compose version
```

Code 4.3 – Vérification de Docker

## 4.2 Configuration de Zabbix

### 4.2.1 Création du Fichier docker-compose.yml

Le fichier docker-compose.yml définit l'ensemble de la stack Zabbix :

```
1 mkdir ~/zabbix && cd ~/zabbix
2 nano docker-compose.yml
```

Code 4.4 – Création du répertoire et du fichier

```
1 version: "3.8"
2
3 services:
4   # Base de données MySQL
5   mysql-server:
6     image: mysql:8.0
7     container_name: zabbix-mysql
8     restart: always
9     environment:
10      MYSQL_DATABASE: zabbix
11      MYSQL_USER: zabbix
12      MYSQL_PASSWORD: zabbix_pwd_secure
13      MYSQL_ROOT_PASSWORD: root_pwd_secure
14     command:
```

```

15     - mysqld
16     - --character-set-server=utf8mb4
17     - --collation-server=utf8mb4_bin
18 volumes:
19     - mysql_data:/var/lib/mysql
20 networks:
21     - zabbix-net
22
23 # Serveur Zabbix
24 zabbix-server:
25     image: zabbix/zabbix-server-mysql:alpine-7.0-latest
26     container_name: zabbix-server
27     restart: always
28     environment:
29         DB_SERVER_HOST: mysql-server
30         MYSQL_DATABASE: zabbix
31         MYSQL_USER: zabbix
32         MYSQL_PASSWORD: zabbix_pwd_secure
33     ports:
34         - "10051:10051"
35     depends_on:
36         - mysql-server
37     networks:
38         - zabbix-net
39
40 # Interface Web Zabbix
41 zabbix-web:
42     image: zabbix/zabbix-web-nginx-mysql:alpine-7.0-latest
43     container_name: zabbix-web
44     restart: always
45     environment:
46         ZBX_SERVER_HOST: zabbix-server
47         DB_SERVER_HOST: mysql-server
48         MYSQL_DATABASE: zabbix
49         MYSQL_USER: zabbix
50         MYSQL_PASSWORD: zabbix_pwd_secure
51         PHP_TZ: Europe/Paris
52     ports:
53         - "80:8080"
54         - "443:8443"
55     depends_on:
56         - zabbix-server
57     networks:
58         - zabbix-net
59
60 # Agent Zabbix local
61 zabbix-agent:
62     image: zabbix/zabbix-agent:alpine-7.0-latest
63     container_name: zabbix-agent
64     restart: always
65     environment:

```

```

66     ZBX_SERVER_HOST: zabbix-server
67     ZBX_HOSTNAME: "Zabbix-Server"
68     depends_on:
69       - zabbix-server
70     networks:
71       - zabbix-net
72
73 volumes:
74     mysql_data:
75
76 networks:
77     zabbix-net:
78       driver: bridge

```

Code 4.5 – Contenu du docker-compose.yml

## 4.3 Lancement des Conteneurs

### 4.3.1 Démarrage de la Stack

```
1 docker compose up -d
```

Code 4.6 – Lancement de Zabbix

```

✓ 1076dc0806dd Pull complete 0.1s
✓ 5483bb473e39 Download complete 0.1s
✓ ec467d4f0f3f Download complete 0.6s
✓ Image zabbix/zabbix-agent:alpine-7.0-latest Pulled 4.1s
✓ c308822633b5 Pull complete 0.3s
✓ 1074333eacdd Pull complete 2.4s
✓ 9e139f95394e Pull complete 0.6s
✓ 636cdd3d9e8d Pull complete 1.7s
✓ 76a53908e8f5 Pull complete 1.9s
✓ 8dc8c9a54070 Pull complete 2.6s
✓ 4f4f070ef54 Pull complete 0.3s
✓ c1ce9fbb06e2 Download complete 0.2s
✓ 4308c440536f Download complete 0.1s
✓ Image zabbix/zabbix-web-nginx-mysql:alpine-7.0-latest Pulled 11.7s
✓ 1074333eacdd Pull complete 1.4s
✓ 4f4f070ef54 Pull complete 0.3s
✓ bda8d130f55a Pull complete 7.6s
✓ 84b9a97d3054 Pull complete 0.6s
✓ f97c72402100 Pull complete 7.5s
✓ 585f1e072df1 Pull complete 10.9s
✓ 24ff955dd120 Download complete 0.6s
✓ 306d5a9e35d2 Download complete 0.4s
✓ Image mysql:8.0 Pulled 20.5s
✓ f93f90ba0229 Pull complete 0.6s
✓ 8858a9417649 Pull complete 0.6s
✓ 26040ca4b0ff Pull complete 0.1s
✓ 276b5a1cf8a7 Pull complete 0.3s
✓ 77e8c0b15ba1 Pull complete 0.2s
✓ ca6b3f314407 Pull complete 0.6s
✓ b55b58084586 Pull complete 0.5s
✓ 2437fb4aadf0 Pull complete 0.6s
✓ 24b098ac9d9e Pull complete 12.1s
✓ 91c53729d27c Pull complete 10.5s
✓ be0e28bac25c Pull complete 0.6s
✓ if4a7e60dc41 Download complete 1.4s
✓ 057be4099f5a Download complete 0.3s
✓ Network zabbix_zabbix-net Created 0.1s
✓ Volume zabbix_mysql_data Created 0.6s
✓ Volume zabbix_zabbix_server_data Created 0.6s
✓ Container zabbix-mysql Healthy 14.0s
✓ Container zabbix-server Created 0.1s
✓ Container zabbix-web Created 0.1s
✓ Container zabbix-agent Created 0.1s
ubuntu@ip-10-0-1-237:~/zabbix$

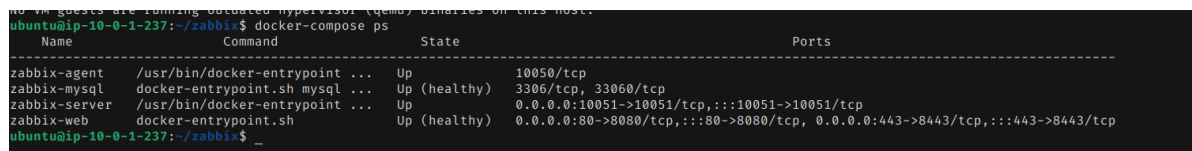
```

FIGURE 4.2 – Lancement des conteneurs Docker

### 4.3.2 Vérification des Conteneurs

```
1 docker compose ps
```

Code 4.7 – Vérification de l'état des conteneurs



```
no VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-10-0-1-237:~/zabbix$ docker-compose ps

```

| Name          | Command                        | State        | Ports                                                                          |
|---------------|--------------------------------|--------------|--------------------------------------------------------------------------------|
| zabbix-agent  | /usr/bin/docker-entrypoint ... | Up           | 10050/tcp                                                                      |
| zabbix-mysql  | docker-entrypoint.sh mysql ... | Up (healthy) | 3306/tcp, 33060/tcp                                                            |
| zabbix-server | /usr/bin/docker-entrypoint ... | Up           | 0.0.0.0:10051->10051/tcp, :::10051->10051/tcp                                  |
| zabbix-web    | docker-entrypoint.sh           | Up (healthy) | 0.0.0.0:80->8080/tcp, :::80->8080/tcp, 0.0.0.0:443->8443/tcp, :::443->8443/tcp |

```
ubuntu@ip-10-0-1-237:~/zabbix$
```

FIGURE 4.3 – Conteneurs Zabbix en cours d'exécution

Les 4 conteneurs doivent être en état `running` :

- **zabbix-mysql** : Base de données MySQL 8.0
- **zabbix-server** : Serveur Zabbix 7.0
- **zabbix-web** : Interface Web Nginx
- **zabbix-agent** : Agent local

## 4.4 Accès à l'Interface Web

### 4.4.1 Connexion à Zabbix

L'interface Web est accessible via l'IP publique du serveur :

**URL** : `http://<IP-PUBLIQUE>`

**Utilisateur** : Admin  
**Mot de passe** : zabbix

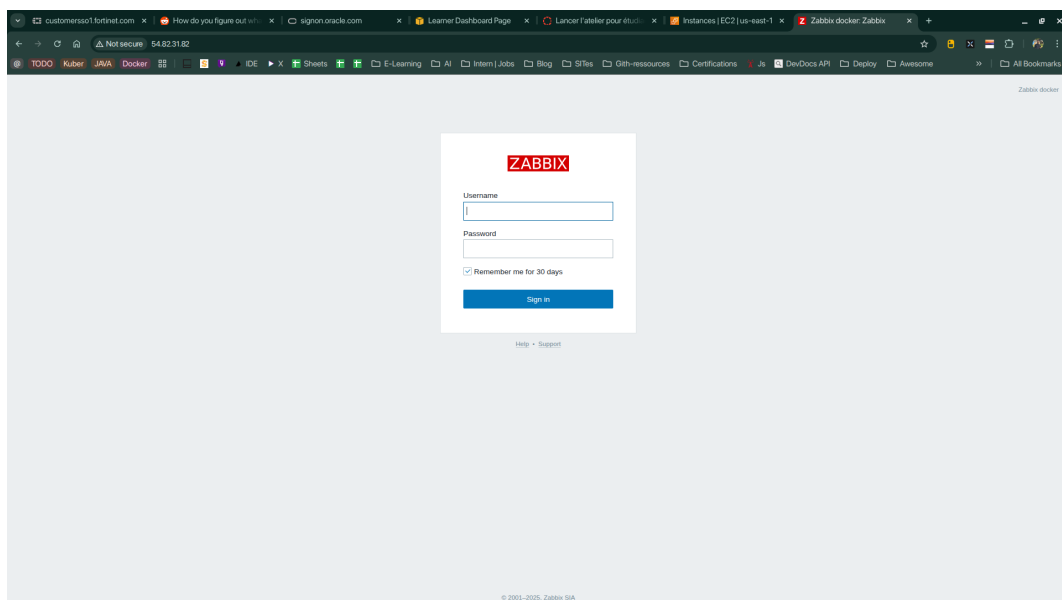


FIGURE 4.4 – Page de connexion Zabbix

### 4.4.2 Dashboard Principal

Après connexion, le tableau de bord principal de Zabbix s'affiche :



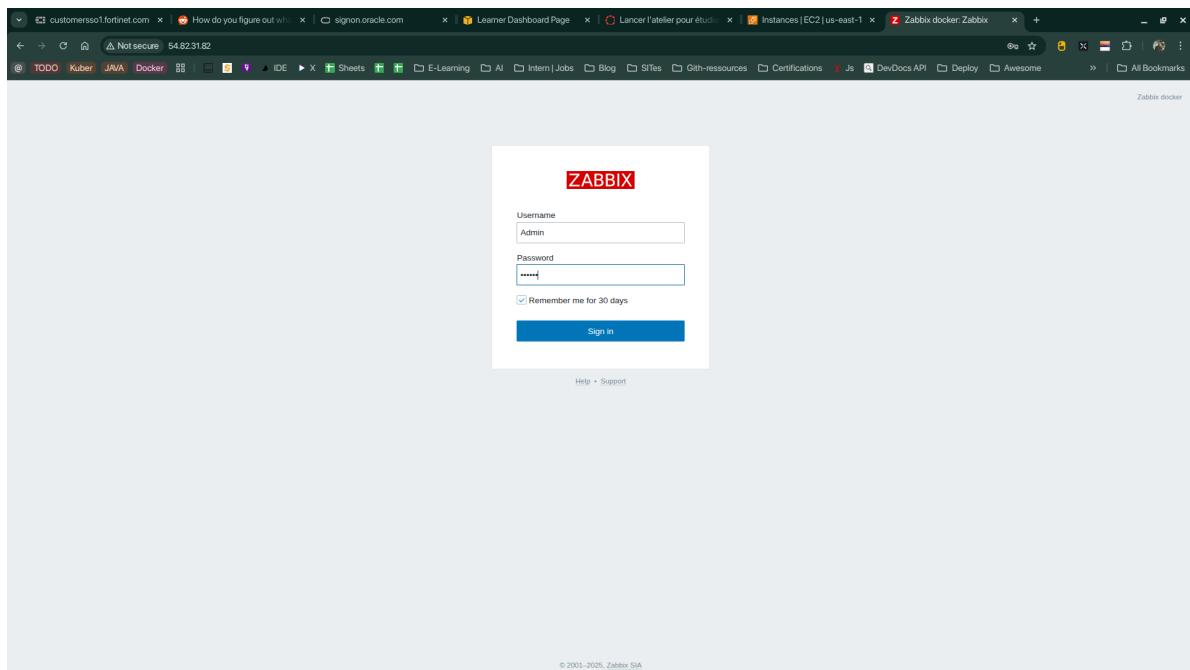


FIGURE 4.5 – Dashboard principal de Zabbix

### 4.5 Architecture des Conteneurs

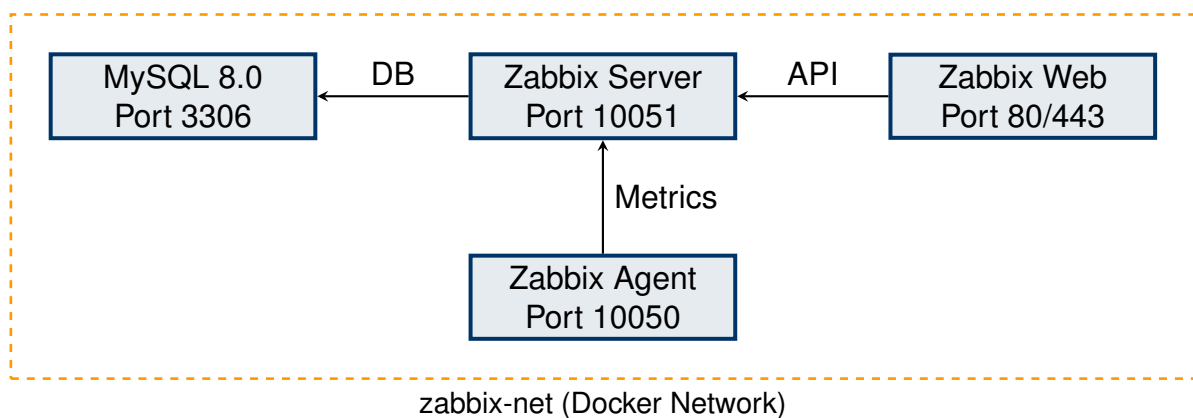


FIGURE 4.6 – Architecture des conteneurs Docker

### 4.6 Commandes Utiles

TABLE 4.1 – Commandes Docker utiles

| Commande                            | Description                    |
|-------------------------------------|--------------------------------|
| <code>docker compose up -d</code>   | Démarrer les conteneurs        |
| <code>docker compose down</code>    | Arrêter les conteneurs         |
| <code>docker compose ps</code>      | Voir l'état des conteneurs     |
| <code>docker compose logs -f</code> | Voir les logs en temps réel    |
| <code>docker compose restart</code> | Redémarrer tous les conteneurs |

# Chapitre 5

## Configuration des Clients (Agents)

### 5.1 Vue d'Ensemble

Les agents Zabbix sont des logiciels installés sur les machines à superviser. Ils collectent les métriques système et les transmettent au serveur Zabbix.

TABLE 5.1 – Modes de communication des agents

| Mode   | Description                                        |
|--------|----------------------------------------------------|
| Passif | Le serveur interroge l'agent (port 10050)          |
| Actif  | L'agent envoie les données au serveur (port 10051) |

### 5.2 Installation sur Linux (Ubuntu)

#### 5.2.1 Connexion au Client Linux

```
1 ssh -i "votre-cle.pem" ubuntu@<IP-CLIENT-LINUX>
```

Code 5.1 – Connexion SSH au client Linux

#### 5.2.2 Installation de l'Agent Zabbix

```
1 # Telecharger le package Zabbix
2 wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/\
3 zabbix-release/zabbix-release_latest_7.0+ubuntu22.04_all.deb
4
5 # Installer le package
6 sudo dpkg -i zabbix-release_latest_7.0+ubuntu22.04_all.deb
7
8 # Mettre a jour et installer l'agent
9 sudo apt update
10 sudo apt install -y zabbix-agent
```

Code 5.2 – Installation de l'agent Zabbix sur Ubuntu

#### 5.2.3 Configuration de l'Agent Linux

Éditer le fichier de configuration :

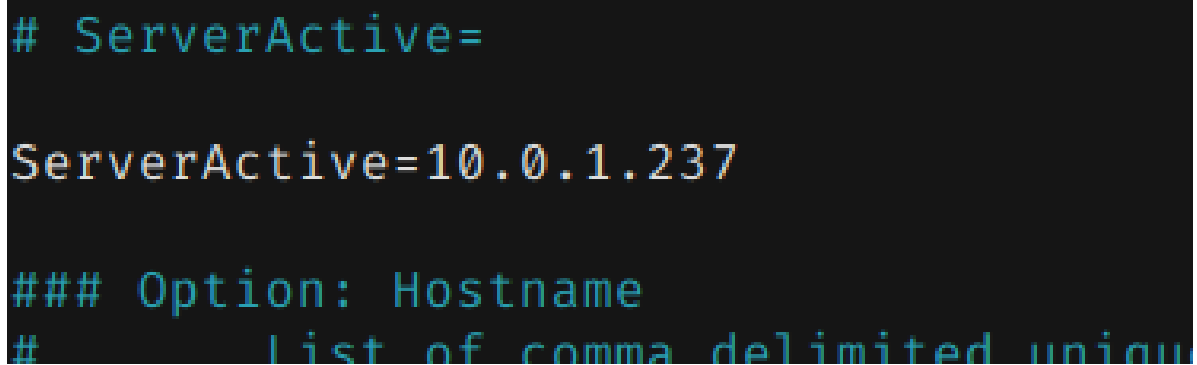
```
1 sudo nano /etc/zabbix/zabbix_agentd.conf
```

Code 5.3 – Édition de la configuration

Modifier les paramètres suivants :

```
1 # Adresse IP du serveur Zabbix (IP privée)
2 Server=10.0.1.X
3
4 # Pour le mode actif
5 ServerActive=10.0.1.X
6
7 # Nom d'hôte (doit correspondre dans Zabbix)
8 Hostname=ELBadry-Client-Linux
9
10 # Autoriser les commandes à distance (optionnel)
11 EnableRemoteCommands=1
```

Code 5.4 – Configuration zabbix\_agentd.conf (Linux)



```
# ServerActive=
ServerActive=10.0.1.237
### Option: Hostname
# List of comma delimited unique
```

FIGURE 5.1 – Configuration de l'agent Zabbix sur Linux

### 5.2.4 Démarrage du Service

```
1 # Redemarrer l'agent
2 sudo systemctl restart zabbix-agent
3
4 # Activer au démarrage
5 sudo systemctl enable zabbix-agent
6
7 # Verifier le statut
8 sudo systemctl status zabbix-agent
```

Code 5.5 – Démarrage de l'agent Linux

### 5.2.5 Vérification de la Connectivité

```
1 # Verifier que l'agent ecoute sur le port 10050
2 sudo netstat -tlnp | grep 10050
```

```
3
4 # Tester la connexion depuis le serveur Zabbix
5 zabbix_get -s <IP-CLIENT-LINUX> -k agent.ping
```

Code 5.6 – Test de connectivité

## 5.3 Installation sur Windows

### 5.3.1 Connexion RDP

Se connecter au client Windows via Remote Desktop :

1. Ouvrir `mstsc.exe`
2. Entrer l'IP publique du client Windows
3. Utiliser les credentials récupérés (Administrator)

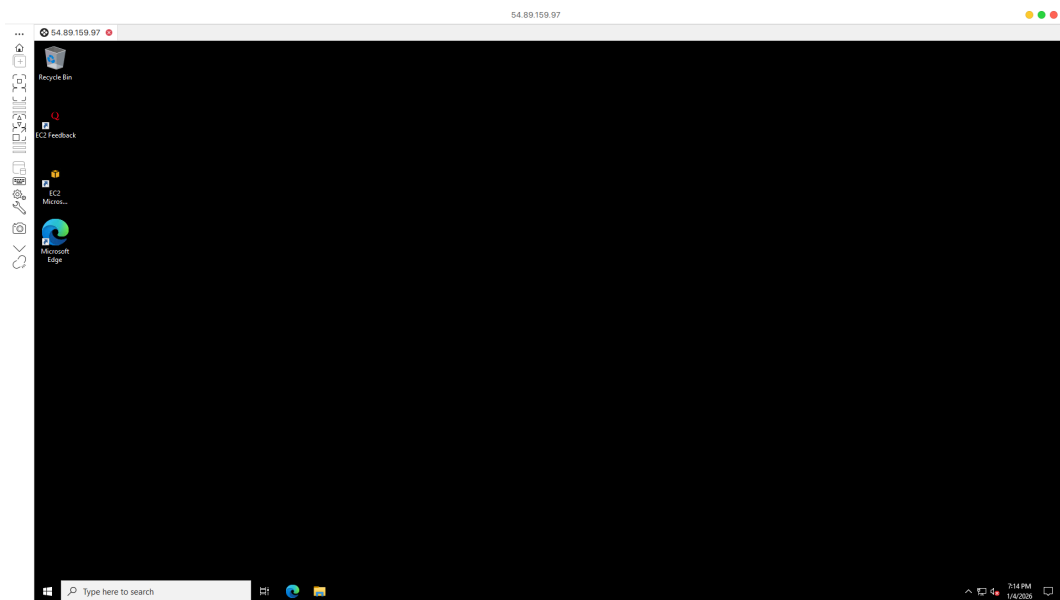


FIGURE 5.2 – Connexion RDP au client Windows

### 5.3.2 Téléchargement de l'Agent

1. Ouvrir le navigateur sur le serveur Windows
2. Aller sur : [https://www.zabbix.com/download\\_agents](https://www.zabbix.com/download_agents)
3. Sélectionner :
  - OS : Windows
  - Version : 7.0 LTS
  - Architecture : AMD64
  - Format : MSI
4. Télécharger le fichier MSI

### 5.3.3 Installation de l'Agent Windows

1. Exécuter le fichier MSI téléchargé
2. Configurer les paramètres :
  - **Zabbix Server IP** : IP privée du serveur (10.0.1.X)
  - **Agent Hostname** : ELBadry-Client-Windows
  - **Listen Port** : 10050
3. Terminer l'installation

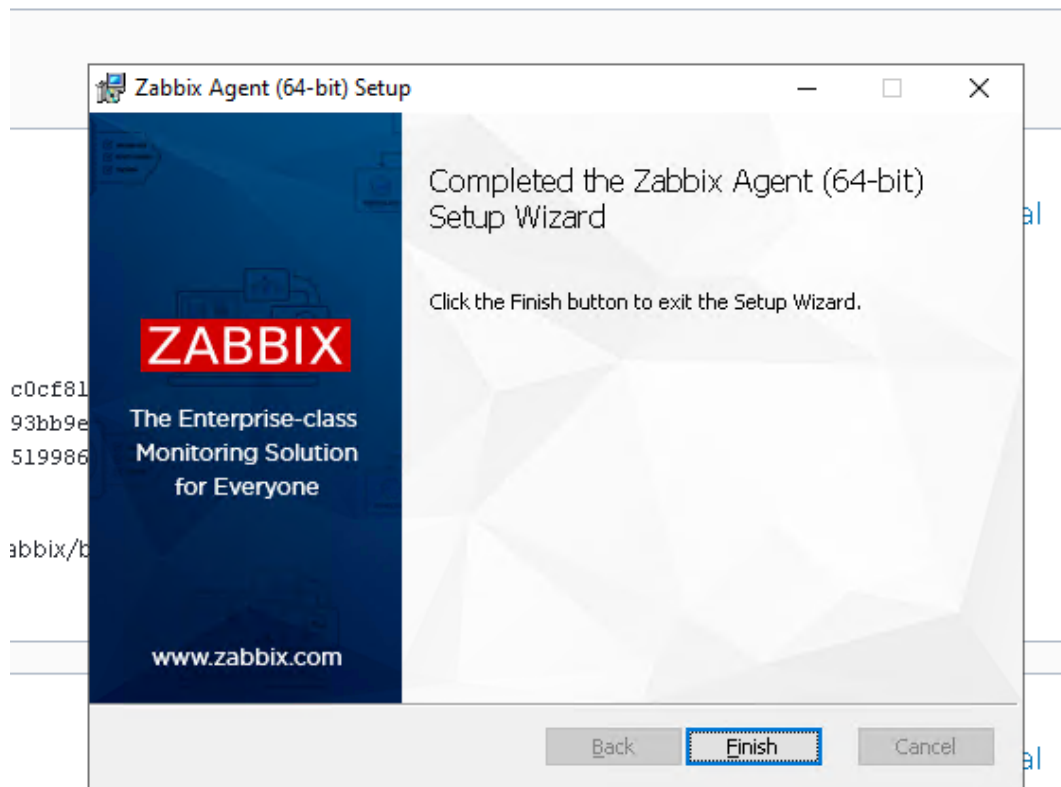


FIGURE 5.3 – Installation de l'agent Zabbix sur Windows

### 5.3.4 Vérification du Service Windows

1. Ouvrir `services.msc`
2. Rechercher "Zabbix Agent"
3. Vérifier que le service est en état "Running"

### 5.3.5 Configuration du Pare-feu Windows

S'assurer que le port 10050 est ouvert :

1. Ouvrir Windows Firewall
2. Créer une règle entrante pour le port TCP 10050
3. Autoriser les connexions

## 5.4 Ajout des Hôtes dans Zabbix

### 5.4.1 Ajouter le Client Linux

1. Dans Zabbix Web : **Data collection** → **Hosts** → **Create host**
2. Configurer :
  - **Host name** : ELBadry-Client-Linux
  - **Groups** : Linux servers
  - **Interfaces** : Agent, IP = 10.0.1.X, Port = 10050
3. Dans l'onglet **Templates** : Ajouter "Linux by Zabbix agent"
4. Cliquer sur **Add**

The screenshot shows the 'Create host' form in Zabbix. The form is titled 'Host' and has tabs for Host, IPMI, Tags, Macros, Inventory, Encryption, and Value mapping. The 'Host' tab is active. The form contains the following fields and controls:

- Host name**: Client-Linux-ELBadry
- Visible name**: Client Linux
- Templates**: type here to search (with a Select button)
- Host groups**: Linux servers (with a Select button)
- Interfaces**: A table with columns Type, IP address, DNS name, Connect to, Port, and Default. The first row shows Agent, 10.0.1.33, (empty), IP, 10050, and a radio button selected for Default.
- Description**: A text area with an 'Add' link.
- Monitored by**: Server, Proxy, Proxy group (radio buttons).
- Enabled**: A checked checkbox.
- Buttons**: Update, Clone, Delete, Cancel.

FIGURE 5.4 – Ajout du client Linux dans Zabbix

### 5.4.2 Ajouter le Client Windows

1. **Data collection** → **Hosts** → **Create host**
2. Configurer :
  - **Host name** : ELBadry-Client-Windows
  - **Groups** : Windows servers (créer si nécessaire)
  - **Interfaces** : Agent, IP = 10.0.1.X, Port = 10050
3. Dans l'onglet **Templates** : Ajouter "Windows by Zabbix agent"
4. Cliquer sur **Add**

## 5.5 Vérification de la Connexion

### 5.5.1 Liste des Hôtes

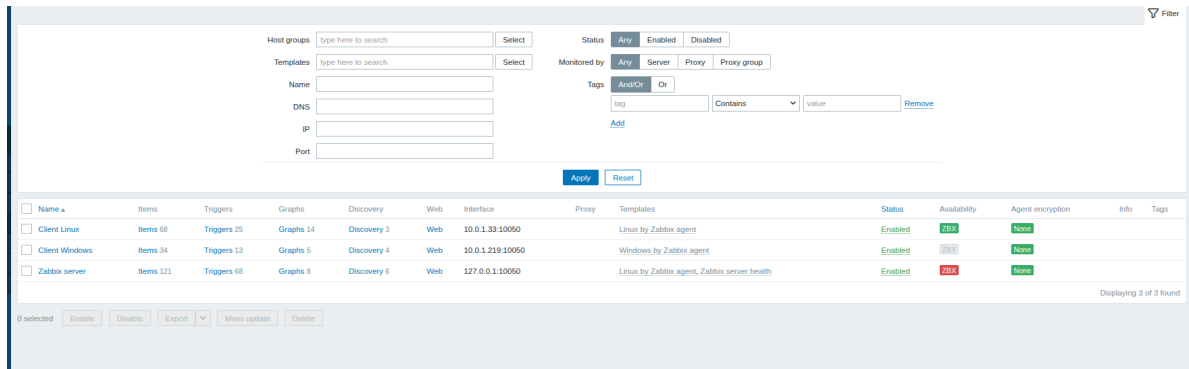


FIGURE 5.5 – Liste des hôtes configurés dans Zabbix

### 5.5.2 Statut des Agents (ZBX Vert)

Le statut "ZBX" vert indique que l'agent est correctement connecté et répond aux requêtes du serveur.

| Name           | Items     | Triggers    | Graphs    | Discovery   | Web | Interface        | Proxy                                       | Templates                                   | Status  | Availability | Agent encryption | Info | Tags |
|----------------|-----------|-------------|-----------|-------------|-----|------------------|---------------------------------------------|---------------------------------------------|---------|--------------|------------------|------|------|
| Client Linux   | Items 68  | Triggers 25 | Graphs 14 | Discovery 3 | Web | 10.0.1.33:10050  | Linux by Zabbix agent                       | Linux by Zabbix agent                       | Enabled | ZBX          | None             |      |      |
| Client Windows | Items 34  | Triggers 13 | Graphs 5  | Discovery 4 | Web | 10.0.1.219:10050 | Windows by Zabbix agent                     | Windows by Zabbix agent                     | Enabled | ZBX          | None             |      |      |
| Zabbix server  | Items 121 | Triggers 68 | Graphs 8  | Discovery 6 | Web | 127.0.0.1:10050  | Linux by Zabbix agent, Zabbix server health | Linux by Zabbix agent, Zabbix server health | Enabled | ZBX          | None             |      |      |

FIGURE 5.6 – Statut ZBX vert - Agents connectés avec succès

## 5.6 Tableau Récapitulatif des Agents

TABLE 5.2 – Configuration des agents

| Propriété     | Client Linux          | Client Windows          |
|---------------|-----------------------|-------------------------|
| Hostname      | ELBadry-Client-Linux  | ELBadry-Client-Windows  |
| IP Privée     | 10.0.1.X              | 10.0.1.X                |
| Port          | 10050                 | 10050                   |
| Template      | Linux by Zabbix agent | Windows by Zabbix agent |
| Version Agent | 7.0                   | 7.0                     |

# Chapitre 6

## Monitoring et Tableaux de Bord

### 6.1 Vue d'Ensemble du Monitoring

Une fois les agents configurés et connectés, Zabbix collecte automatiquement les métriques des hôtes supervisés. Cette section présente les fonctionnalités de monitoring et de visualisation.

### 6.2 Dashboard Principal

Le tableau de bord principal de Zabbix affiche une vue synthétique de l'état de l'infrastructure :

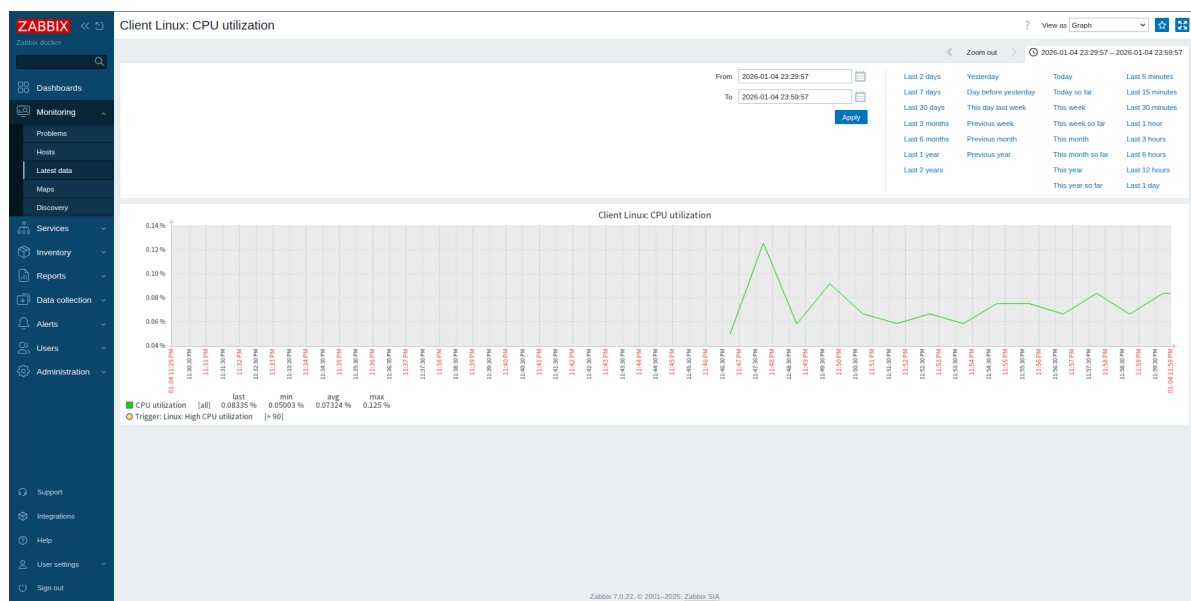


FIGURE 6.1 – Dashboard de monitoring Zabbix

### 6.3 Métriques Collectées

#### 6.3.1 Métriques Système Linux

Le template "Linux by Zabbix agent" collecte automatiquement :



TABLE 6.1 – Métriques Linux collectées

| Catégorie | Métriques                                             |
|-----------|-------------------------------------------------------|
| CPU       | Utilisation, load average, temps utilisateur/-système |
| Mémoire   | RAM utilisée/libre, swap, cache                       |
| Disque    | Espace utilisé, I/O, inodes                           |
| Réseau    | Trafic entrant/sortant, erreurs, paquets              |
| Processus | Nombre de processus, zombies                          |
| Système   | Uptime, hostname, version OS                          |

### 6.3.2 Métriques Système Windows

Le template "Windows by Zabbix agent" collecte :

TABLE 6.2 – Métriques Windows collectées

| Catégorie  | Métriques                                      |
|------------|------------------------------------------------|
| CPU        | Utilisation par cœur, temps processeur         |
| Mémoire    | Mémoire physique, mémoire virtuelle, page file |
| Disque     | Espace disque, temps de lecture/écriture       |
| Réseau     | Bande passante, erreurs d'interface            |
| Services   | État des services Windows                      |
| Événements | Logs Windows (Système, Application)            |

## 6.4 Test de Charge (Stress Test)

### 6.4.1 Objectif

Pour démontrer la capacité de Zabbix à détecter les anomalies, nous effectuons un test de charge CPU sur le client Linux.

### 6.4.2 Exécution du Test

```
1 # Installation de l'outil stress
2 sudo apt install -y stress
3
4 # Lancement du stress test CPU (4 coeurs pendant 60s)
5 stress --cpu 4 --timeout 60s
```

Code 6.1 – Installation et exécution de stress

```
ubuntu@ip-10-0-1-33:~$ stress --cpu 4 --timeout 60s
stress: info: [2034] dispatching hogs: 4 cpu, 0 io, 0 vm, 0 hdd
stress: info: [2034] successful run completed in 60s
ubuntu@ip-10-0-1-33:~$ stress --cpu 2 --timeout 60s
stress: info: [2063] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [2063] successful run completed in 60s
ubuntu@ip-10-0-1-33:~$ client_loop: send disconnect: Broken pipe
└─elbadry_@moelbadry ~/Dev/Other/Cyber-project
```

FIGURE 6.2 – Exécution de la commande stress

### 6.4.3 Observation dans Zabbix

Le pic de CPU est immédiatement visible dans les graphiques Zabbix :

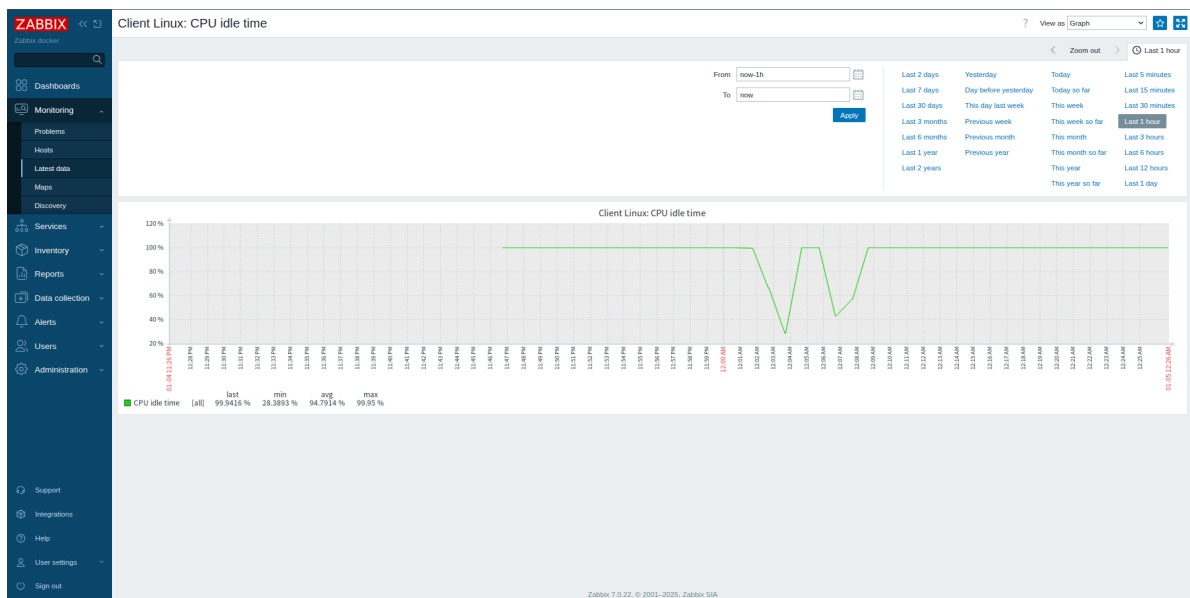


FIGURE 6.3 – Pic de CPU détecté suite au stress test

## 6.5 Gestion des Alertes

### 6.5.1 Problèmes Détectés

Zabbix déclenche automatiquement des alertes basées sur les triggers prédéfinis :

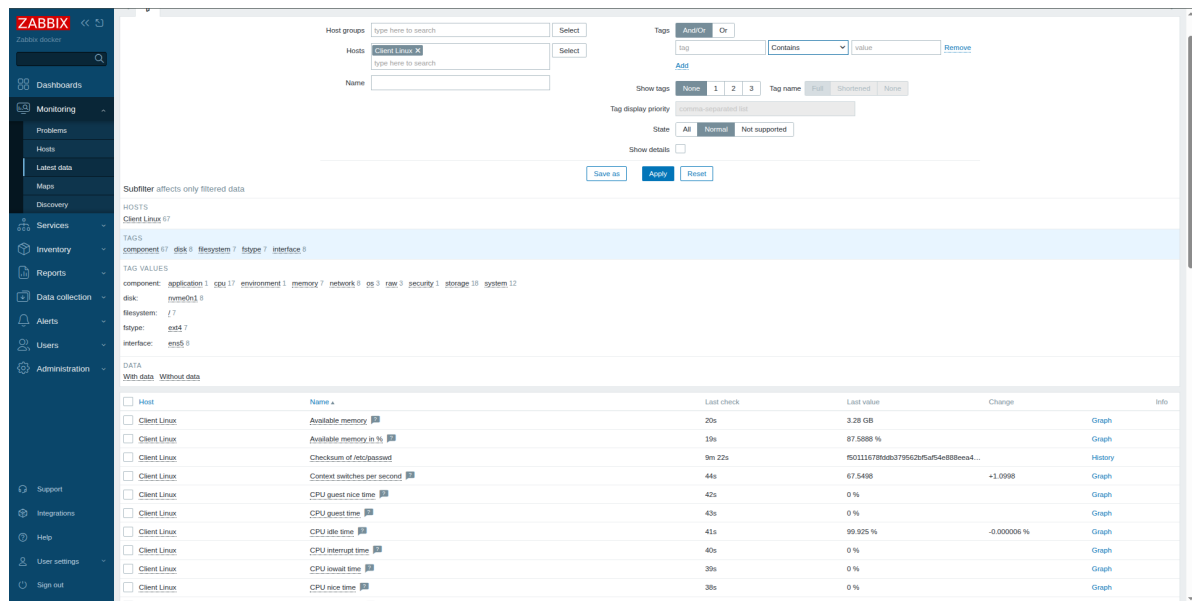


FIGURE 6.4 – Vue des problèmes actifs dans Zabbix

### 6.5.2 Liste des Problèmes

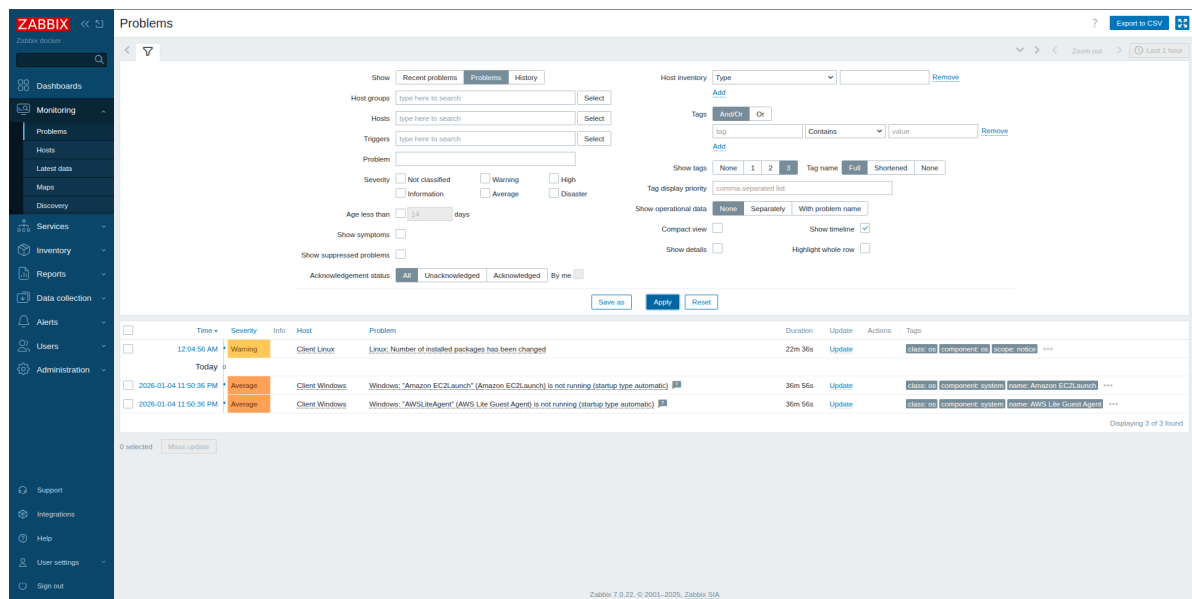


FIGURE 6.5 – Liste détaillée des problèmes

### 6.5.3 Historique des Événements

L'historique complet des événements est disponible :

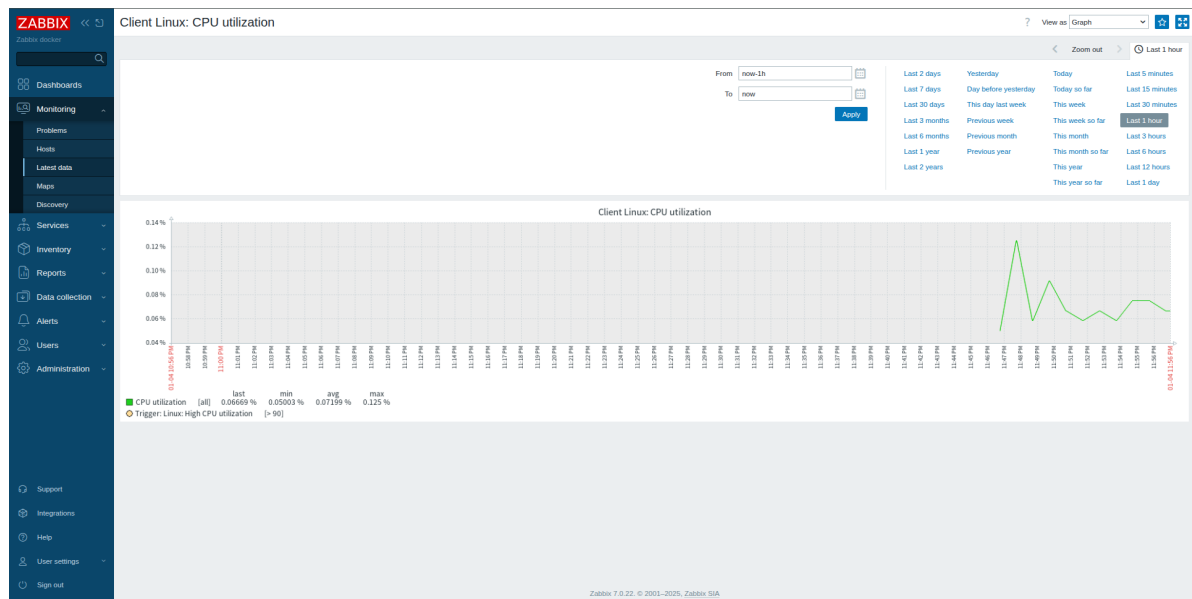


FIGURE 6.6 – Historique des événements Zabbix

## 6.6 Configuration des Triggers

### 6.6.1 Liste des Triggers

Les triggers définissent les conditions de déclenchement des alertes :

The screenshot shows the Zabbix web interface for monitoring 'Client Linux: FS [/]: Space: Available'. The main area features a table with 11 rows of data. The columns are 'Timestamp' and 'FS [/] Space: Available'. The data shows a constant value of 17826637769 for the 'FS [/] Space: Available' column across all timestamps. The interface includes a sidebar with navigation options and a top right section with time range filters and a 'View as' dropdown set to 'Values'.

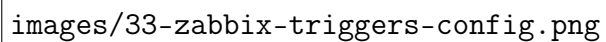
| Timestamp              | FS [/] Space: Available |
|------------------------|-------------------------|
| 2026-01-04 11:58:04 PM | 17826637769             |
| 2026-01-04 11:57:04 PM | 17826637769             |
| 2026-01-04 11:56:04 PM | 17826637769             |
| 2026-01-04 11:55:04 PM | 17826637769             |
| 2026-01-04 11:54:04 PM | 17826637769             |
| 2026-01-04 11:53:04 PM | 17826637769             |
| 2026-01-04 11:52:04 PM | 17826637769             |
| 2026-01-04 11:51:04 PM | 17826637769             |
| 2026-01-04 11:50:04 PM | 17826637769             |
| 2026-01-04 11:49:04 PM | 17826637769             |
| 2026-01-04 11:48:04 PM | 17826637769             |

FIGURE 6.7 – Liste des triggers configurés

## 6.6.2 Exemples de Triggers

TABLE 6.3 – Exemples de triggers Zabbix

| Trigger                 | Sévérité | Condition                    |
|-------------------------|----------|------------------------------|
| High CPU utilization    | Warning  | CPU > 80% pendant 5 min      |
| High memory utilization | Warning  | RAM > 90%                    |
| Disk space critical     | High     | Espace disque < 10%          |
| Agent unreachable       | Disaster | Pas de réponse pendant 3 min |

The image shows a screenshot of the Zabbix web interface for configuring a trigger. It displays the 'Trigger' tab with a list of triggers. The trigger 'Agent unreachable' is highlighted, showing its severity as 'Disaster' and its condition as 'Pas de réponse pendant 3 min'. The interface includes a search bar, a list of triggers, and a detailed view of the selected trigger.

images/33-zabbix-triggers-config.png

FIGURE 6.8 – Configuration d'un trigger

## 6.7 Visualisation des Données

### 6.7.1 Latest Data

La section "Latest data" affiche les dernières valeurs collectées pour chaque métrique :

- **Monitoring** → **Latest data**
- Filtrer par hôte ou groupe
- Voir les valeurs en temps réel
- Accéder aux graphiques historiques

### 6.7.2 Graphiques

Zabbix génère automatiquement des graphiques pour visualiser l'évolution des métriques dans le temps :

- Graphiques CPU (utilisation, load)
- Graphiques mémoire (utilisée, libre)
- Graphiques réseau (trafic in/out)
- Graphiques disque (espace, I/O)

## 6.8 Récapitulatif du Monitoring

TABLE 6.4 – État final du monitoring

| Hôte                   | Statut | Template                | Items |
|------------------------|--------|-------------------------|-------|
| Zabbix-Server          | ZBX    | Linux by Zabbix agent   | 150   |
| ELBadry-Client-Linux   | ZBX    | Linux by Zabbix agent   | 150   |
| ELBadry-Client-Windows | ZBX    | Windows by Zabbix agent | 200   |

## 6.9 Points Clés

- Les 3 hôtes sont supervisés avec succès (ZBX vert)
- Les métriques sont collectées en temps réel
- Le stress test a déclenché les alertes appropriées
- Les graphiques affichent l'historique des performances
- Les triggers fonctionnent correctement

# Chapitre 7

## Conclusion

### 7.1 Synthèse du Projet

Ce projet a permis de mettre en œuvre avec succès une **infrastructure cloud de supervision centralisée** sur AWS. L'ensemble des objectifs fixés ont été atteints :

TABLE 7.1 – Objectifs atteints

| Objectif                                      | Statut |
|-----------------------------------------------|--------|
| Création du VPC et de l'infrastructure réseau |        |
| Déploiement de 3 instances EC2                |        |
| Installation de Docker sur le serveur         |        |
| Déploiement de Zabbix conteneurisé            |        |
| Configuration de l'agent Linux                |        |
| Configuration de l'agent Windows              |        |
| Monitoring opérationnel (ZBX vert)            |        |
| Test d'alertes fonctionnel                    |        |

### 7.2 Compétences Acquises

#### 7.2.1 Amazon Web Services

- Configuration de VPC, subnets et Internet Gateway
- Gestion des Security Groups et règles de pare-feu
- Déploiement et gestion d'instances EC2
- Utilisation des différents types d'AMI (Ubuntu, Windows Server)
- Connexion SSH et RDP aux instances

#### 7.2.2 Docker et Conteneurisation

- Installation et configuration de Docker Engine
- Rédaction de fichiers docker-compose.yml
- Gestion des conteneurs, volumes et réseaux Docker
- Déploiement d'applications multi-conteneurs

### 7.2.3 Zabbix et Monitoring

- Architecture serveur/agent de Zabbix
- Configuration des agents Linux et Windows
- Utilisation des templates de monitoring
- Configuration des triggers et alertes
- Interprétation des métriques et graphiques

## 7.3 Difficultés Rencontrées

### 7.3.1 Problèmes et Solutions

TABLE 7.2 – Difficultés et solutions apportées

| Difficulté                              | Solution                                                          |
|-----------------------------------------|-------------------------------------------------------------------|
| Temps de démarrage des conteneurs MySQL | Utilisation de healthcheck et depends_on dans docker-compose      |
| Communication agent/serveur bloquée     | Vérification des Security Groups et ouverture du port 10050       |
| Pare-feu Windows bloquant l'agent       | Création d'une règle entrante pour le port 10050                  |
| Session Learner Lab expirée             | Redémarrage des conteneurs avec docker compose up -d              |
| Récupération du mot de passe Windows    | Utilisation de la fonction "Get Windows Password" avec la clé PEM |

## 7.4 Limitations du Learner Lab

Durant ce projet, plusieurs limitations du Learner Lab AWS Academy ont été identifiées :

- **Types d'instances** : Limités à t3.medium et t3.large
- **Région** : Obligatoirement us-east-1 (N. Virginia)
- **Budget** : Limité à environ 50\$ de crédits
- **Sessions** : Arrêt automatique après un certain temps d'inactivité
- **Services** : Certains services AWS avancés non disponibles

## 7.5 Améliorations Possibles

Pour aller plus loin, plusieurs améliorations pourraient être apportées :

1. **Haute disponibilité** : Déployer Zabbix en cluster avec réplication
2. **HTTPS** : Configurer un certificat SSL pour l'interface web
3. **Notifications** : Configurer les alertes par email ou Slack



4. **Automatisation** : Utiliser Terraform pour l'infrastructure as code
5. **Monitoring avancé** : Ajouter la supervision d'applications (MySQL, Nginx)
6. **Dashboards personnalisés** : Créer des tableaux de bord adaptés aux besoins

### 7.6 Perspectives

Ce projet constitue une base solide pour :

- La mise en place de solutions de monitoring en entreprise
- L'apprentissage approfondi des services AWS
- La maîtrise de Docker et de l'orchestration de conteneurs
- La préparation aux certifications AWS et DevOps

### 7.7 Conclusion Générale

La réalisation de ce projet a permis d'acquérir une expérience pratique significative dans le domaine du cloud computing et du monitoring d'infrastructure. L'utilisation combinée d'AWS, Docker et Zabbix offre une solution robuste et évolutive pour la supervision d'environnements hybrides.

#### Ressources du Projet

Dépôt GitHub :

<https://github.com/VOTRE-USERNAME/aws-zabbix-monitoring>

Documentation complète disponible dans le README